

Module – 4:

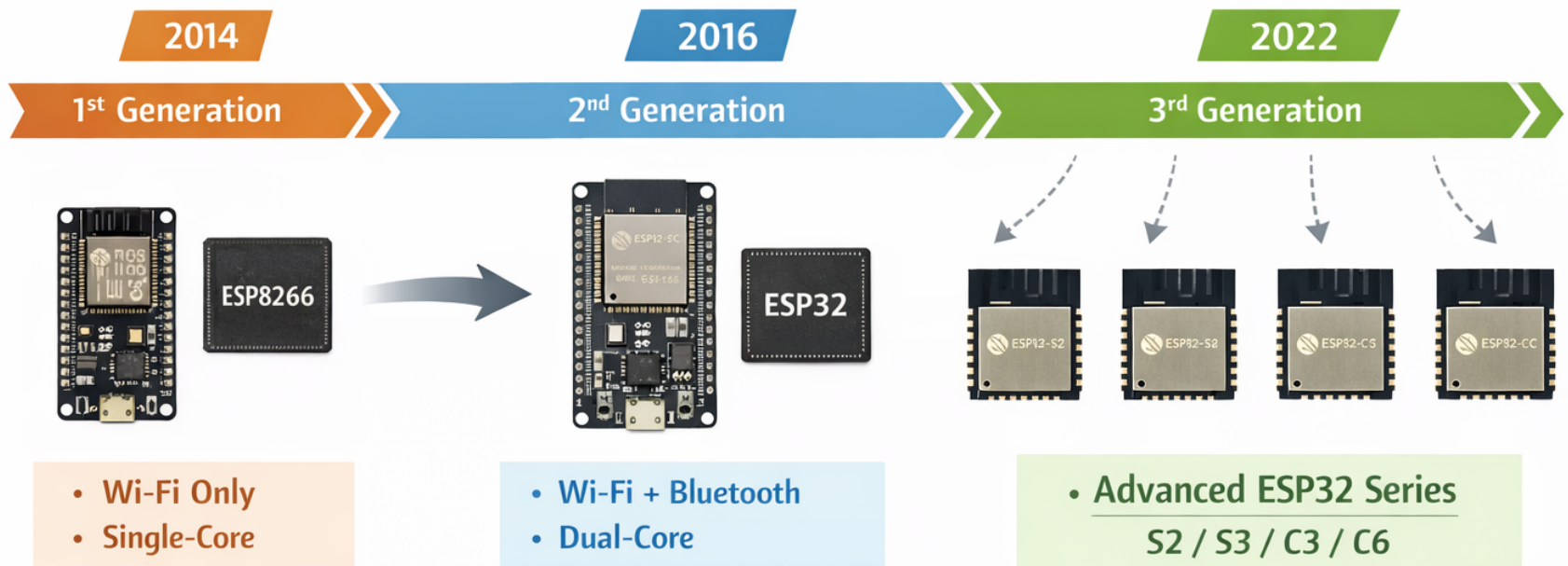
ESP32 Micro-Python

Programming

Introduction

ESP32 comes with an on-chip 32-bit microcontroller with integrated Wi-Fi + Bluetooth + BLE features that targets a wide range of applications. It is a series of low-power and low-cost developed by [Espressif Systems](#).

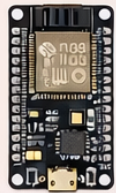
Evolution of Espressif Chips



Comparison of ESP32 Boards

ESP8266 vs ESP32 vs Advanced ESP32 Series

ESP8266



ESP32



Advanced ESP32 Series



ESP32-S2 ESP32-S3 ESP32-C3 ESP32-C6

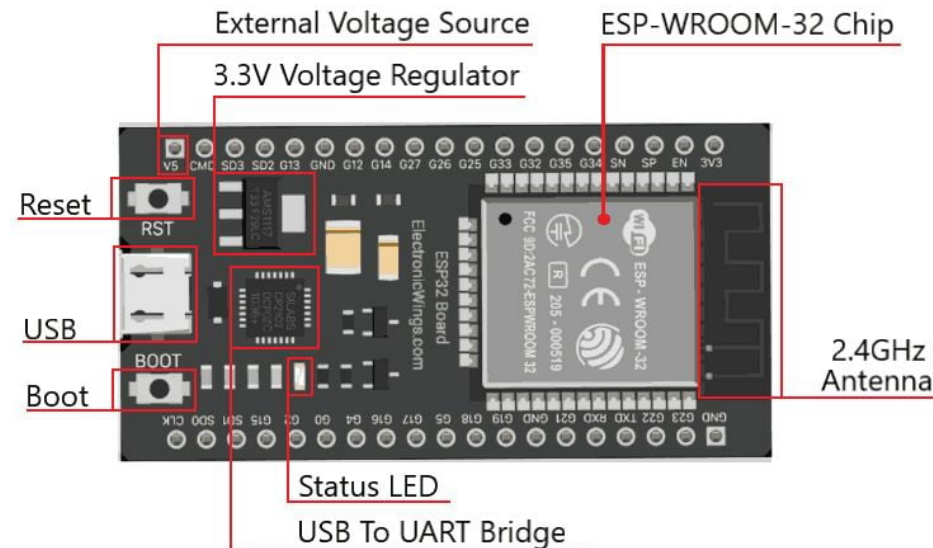
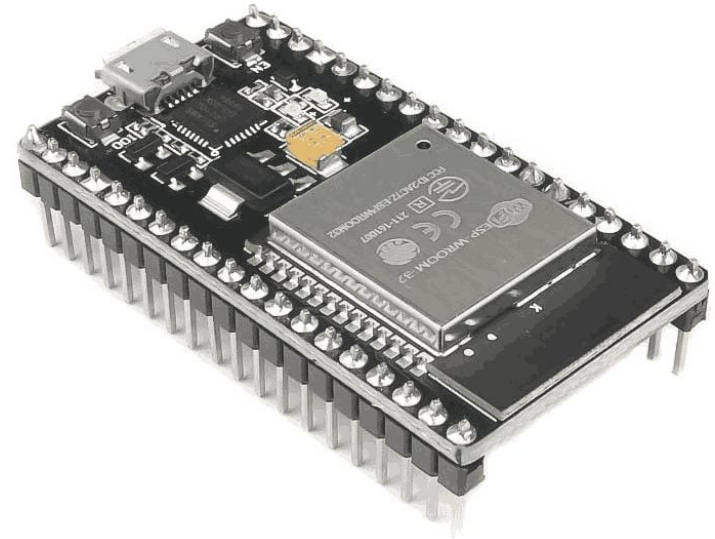
Processor	32-bit
Processor	Single-Core 80 MHz
Wi-Fi	Wi-Fi Only
Bluetooth	✗ No
GPIO Pins	11
RAM	80 KB + 32 KB
Peripherals	ADC, PWM, SPI, I2C
RAM	80 KB + 32 KB
Peripherals	ADC, PWM, SPI, I2C
Power Consumption	Lower

32-bit
✓ Dual-Core 160-240 MHz
✓ Wi-Fi + Bluetooth
✓ Yes (classic + BLE)
30+
Up to 520 KB
ADC, DAC, SPI, I2C, CAN, Touch Sensor, More
Up to 520 KB
ADC, DAC, SPI, I2C, CAN, Touch Sensor, More
Lower in Sleep Mode

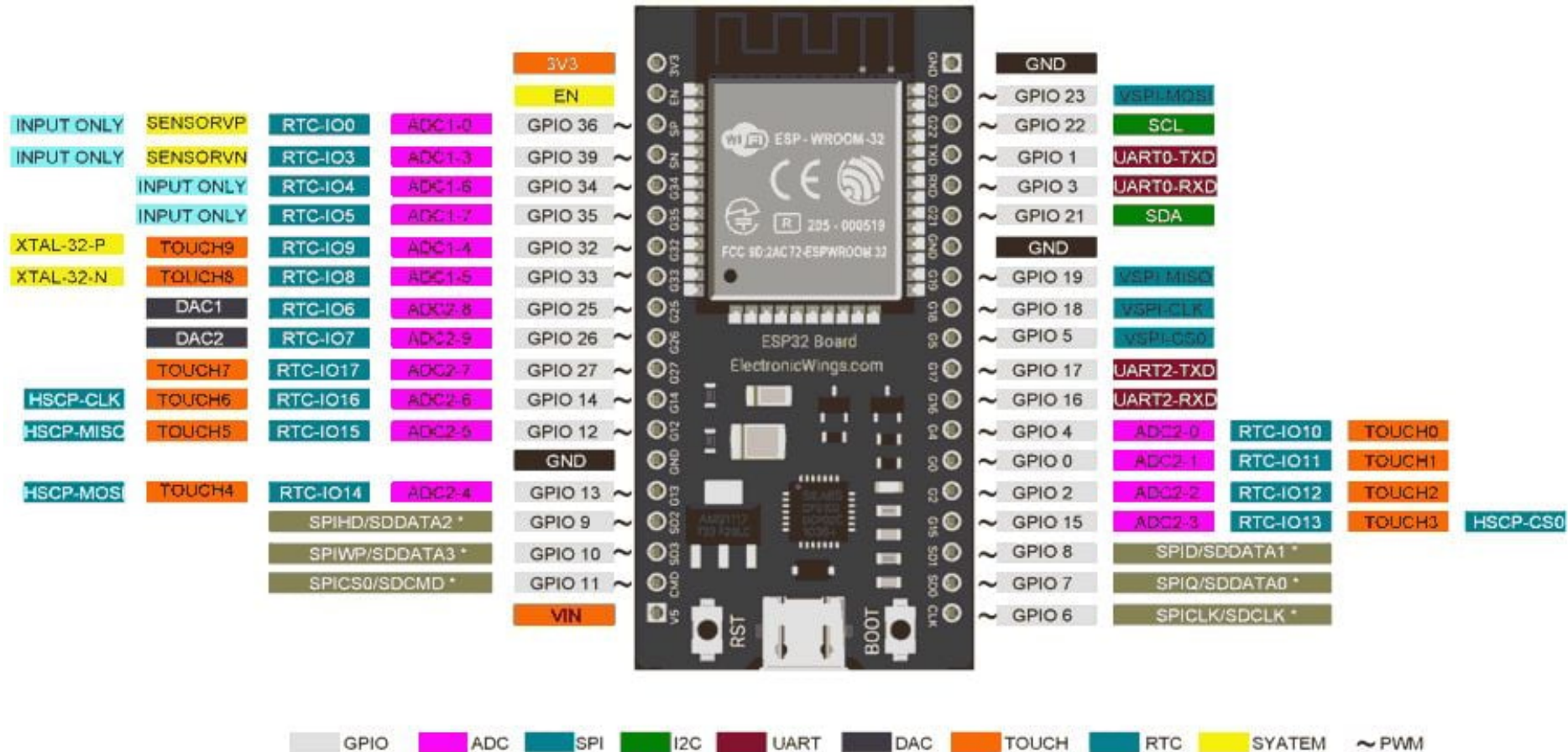
322-bit / 32-bit / RISC-V
✓ Single-Core, Dual-Core, RISC-V
✓ Wi-Fi 4, Wi-Fi 6, Matter, Thread, Zigbee
✓ Yes (classic + BLE)
15-45+
Up to 512 KB SRAM + up to 8 MB PSRAM
Improved ADC, DAC, SPI, I2C, USB, LCD Interface, Zigbee / Matter / Thread, More
15-KB+ SRAM
Improved ADC, DAC, SPI, I2C, USB, LCD Interface, Zigbee / Matter / Thread, More
Lower in Sleep Mode

Features of ESP32-WROOM-32 DevKit

- 32-bit dual-core Xtensa LX6 processor (up to 240 MHz)
- Built-in Wi-Fi (802.11 b/g/n) and Bluetooth /BLE
- 520 KB SRAM and typically 4 MB flash memory
- 30+ GPIO pins with digital I/O capability
- 12-bit ADC, 2 DAC channels, PWM, and capacitive touch sensing
- Communication interfaces: UART, SPI, I²C, I²S, CAN
- USB-to-Serial interface for easy programming
- Supports Arduino IDE, MicroPython, and ESP-IDF
- Low-power modes for IoT and battery applications



ESP32-WROOM-32 : Pin Configuration



ESP32 DevKit Pin Overview

Category	Example Pins / Buttons	Purpose
Power & Control	VIN, 3V3, GND, EN (Reset), BOOT (GPIO0)	Supplies power to the board. EN resets the ESP32. BOOT places the ESP32 into firmware flashing mode during programming.
Digital GPIO	GPIO0–GPIO33	General-purpose digital input/output pins used to control LEDs, relays, motors, switches, etc.
Communication Interfaces	UART (GPIO1,3,16,17), I ² C (GPIO21,22), SPI (GPIO23,19,18,5)	Enables communication with external devices, such as sensors, displays, SD cards, GPS modules, and other microcontrollers.
Analog Functions	ADC (GPIO32–GPIO39), DAC (GPIO25,26)	Used to read analog sensor signals (<i>temperature, light, etc.</i>) using ADC and generate analog voltage signals using DAC.
Special Features	Touch Pins (GPIO0,2,4,12–15,27)	Capacitive touch sensing that allows the ESP32 to detect human touch for buttons, panels, or smart interfaces.

- ✓ -30 programmable GPIO pins
- ✓ Built-in Wi-Fi + Bluetooth
- ✓ Supports UART, SPI, I²C, ADC, DAC, Touch sensing
- ✓ Designed for IoT and embedded applications

Programming Languages & Workflows

Language	Category	Primary Tools	Workflow	Best For
C++ / C	Compiled	Arduino IDE, PlatformIO	Write code → Compile → Flash binary	Hobbyists & IoT
C / C++	Native SDK	ESP-IDF (Espressif)	Write code → Build via CLI/Extension → Flash	Professional Firmware
Python	Interpreted	MicroPython, Thonny	Flash Firmware once → Upload .py scripts	Rapid Prototyping
JavaScript	Interpreted	Espruino, Moddable	Flash Interpreter → Send JS via Web IDE	Web Developers
Lua	Interpreted	NodeMCU Firmware	Flash Firmware → Send commands over Serial	Event-driven apps
Rust	Compiled	Rust (esp-rs)	Write code → Cross-compile → Flash	Memory-safe systems

ESP32 Devkit MicroPython Workflow

The MicroPython workflow for an ESP32 NodeMCU differs from traditional Arduino programming because you install a "Python engine" (firmware) on the board first, then upload and run scripts without needing to re-compile.

Phase 1: Initial Setup (One-Time)

Before you can run Python code, you must "prepare" the ESP32 hardware.

1.Download Firmware: Get the latest stable **.bin** file from the [official MicroPython Downloads page](#).

2.Install an IDE: Use a beginner-friendly tool like Thonny IDE or Mu Editor.

3.Flash the Board:

1. Connect your ESP32 via USB.
2. In Thonny, go to Tools > Options > Interpreter and select **MicroPython (ESP32)**.
3. Click **Install or update firmware**, select your port, and browse for the downloaded **.bin** file.
4. **Note:** You may need to hold the **BOOT** button on your board when the "Erasing" or "Connecting" message appears.

ESP32 Devkit MicroPython Workflow

Phase 2: The Development Cycle

Once flashed, you can write and test code instantly.

- **REPL (Interactive Prompt):** The "Shell" at the bottom of your IDE lets you type Python commands (like `print("Hello")`) and see them run on the [ESP32](#) immediately—no uploading required.
- **Running Scripts:** Write your code in the editor and click the **Run** button. This sends the code to the board's RAM to execute, but it won't stay there if you unplug the power.
- **Saving to Device:** To make code permanent, go to File > Save As and choose **MicroPython device**.

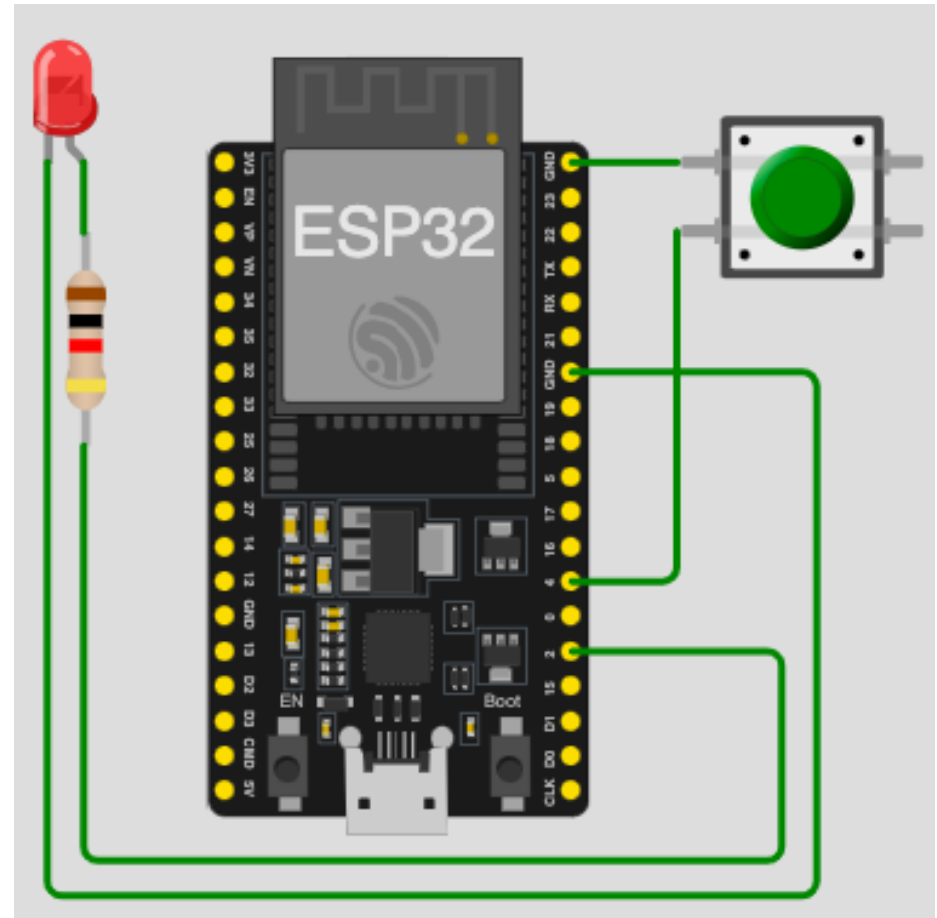
Phase 3: Automated Booting

MicroPython looks for two specific files on the internal storage when it starts up:

- 1.boot.py:** Runs first. Used for low-level configuration like connecting to Wi-Fi. It is usually created automatically during flashing.
- 2.main.py:** Runs second. **This is where your main project code goes.** If you save your script as `main.py` on the device, it will run automatically every time the board is powered on.

Example-1 : Button Controlled LED

```
1  from machine import Pin
2  import time
3
4  led = Pin(2, Pin.OUT)
5  button = Pin(4, Pin.IN, Pin.PULL_UP)
6
7  while True:
8
9      if button.value() == 0:
10         led.value(1)
11         time.sleep(0.5)
12         led.value(0)
13         time.sleep(0.5)
14
15     else:
16         led.value(0)
```



<https://wokwi.com/projects/458290310831866881>

Example-2 : Potentiometer Controlled Servo

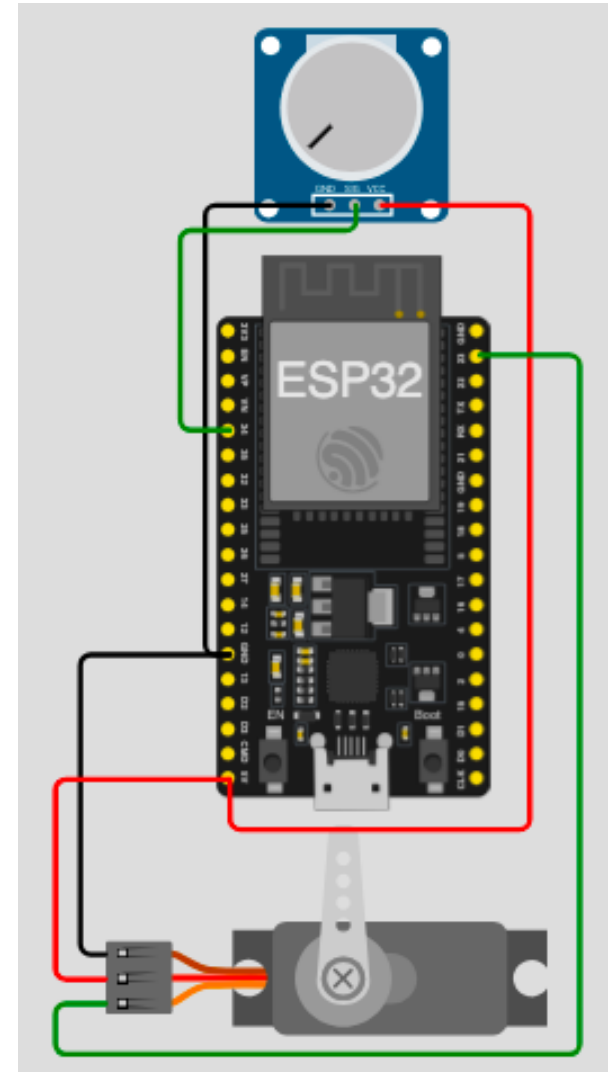
Potentiometer

3.3V ---- Pot ---- GND
|
GPIO34 (ADC)

Servo Motor

Red → 5V / VIN
Brown → GND
Yellow/Orange → GPIO23 (PWM)

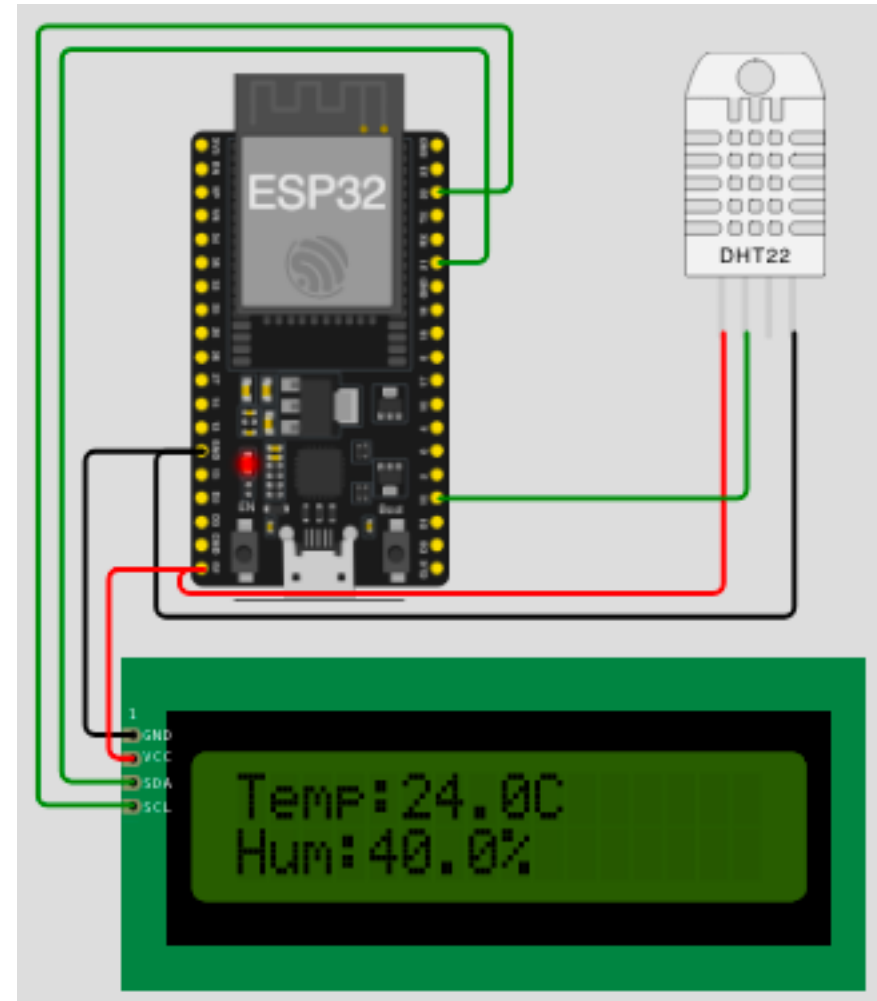
```
1  from machine import Pin, PWM, ADC
2  import time
3
4  pot = ADC(Pin(34))
5  pot.atten(ADC.ATTN_11DB)
6
7  servo = PWM(Pin(23), freq=50)
8
9  while True:
10
11     value = pot.read()      # 0-4095
12
13     duty = int(40 + (value/4095)*75)
14
15     servo.duty(duty)
16
17     time.sleep(0.1)
```



<https://wokwi.com/projects/458306539911434241>

Example-3 : DHT22 & LCD Interfacing

```
1  from machine import Pin, I2C
2  from i2c_lcd import I2cLcd
3  import dht
4  import time
5
6  i2c = I2C(0, scl=Pin(22), sda=Pin(21))
7  lcd = I2cLcd(i2c, 0x27, 2, 16)
8
9  sensor = dht.DHT22(Pin(15))
10
11 while True:
12
13     sensor.measure()
14
15     t = sensor.temperature()
16     h = sensor.humidity()
17
18     lcd.clear()
19     lcd.putstr("Temp:{}".format(t))
20
21     lcd.move_to(0,1)
22     lcd.putstr("Hum:{}".format(h))
23
24     time.sleep(2)
```



<https://wokwi.com/projects/458291704077178881>

Example-4 : Internet Time Through Wi-Fi

```
1  import network
2  import urequests
3  import time
4
5  wifi = network.WLAN(network.STA_IF)
6  wifi.active(True)
7  wifi.connect("Wokwi-GUEST","")
8
9  while not wifi.isconnected():
10     pass
11
12     print("WiFi Connected")
13
14  while True:
15     response = urequests.get("http://worldtimeapi.org/api/timezone/Asia/Kolkata")
16
17     data = response.json()
18     current_time = data["datetime"].split("T")[1].split(".")[0]
19     print("Time:", current_time)
20
21     response.close()
22
23     time.sleep(5)
```

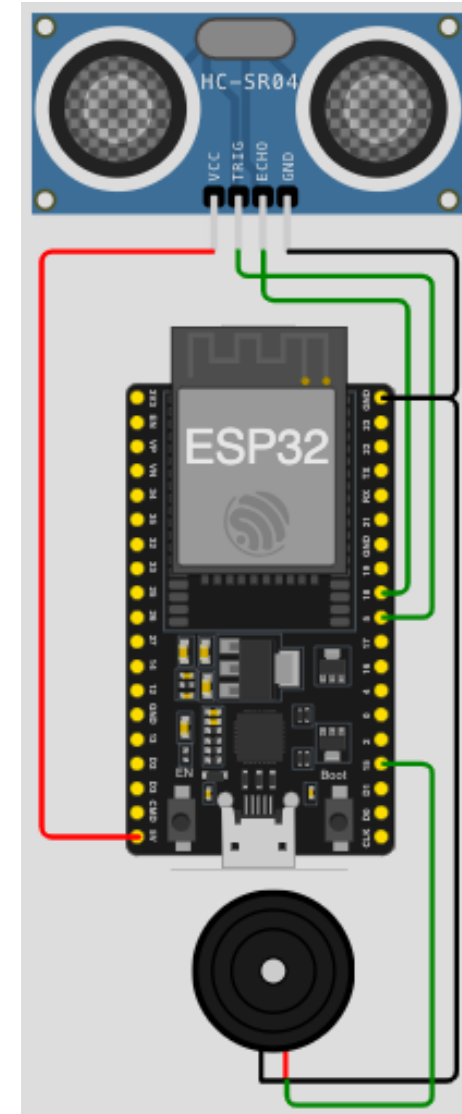
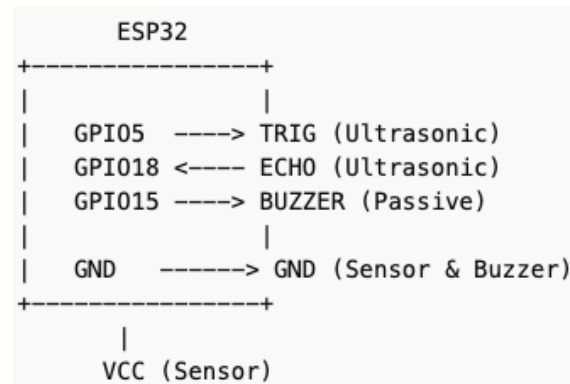


```
WiFi Connected
Time: 22:32:07
Time: 22:32:19
Time: 22:32:31
Time: 22:32:42
Time: 22:32:54
```

<https://wokwi.com/projects/458306833087037441>

Example-5 : Proximity Alert System

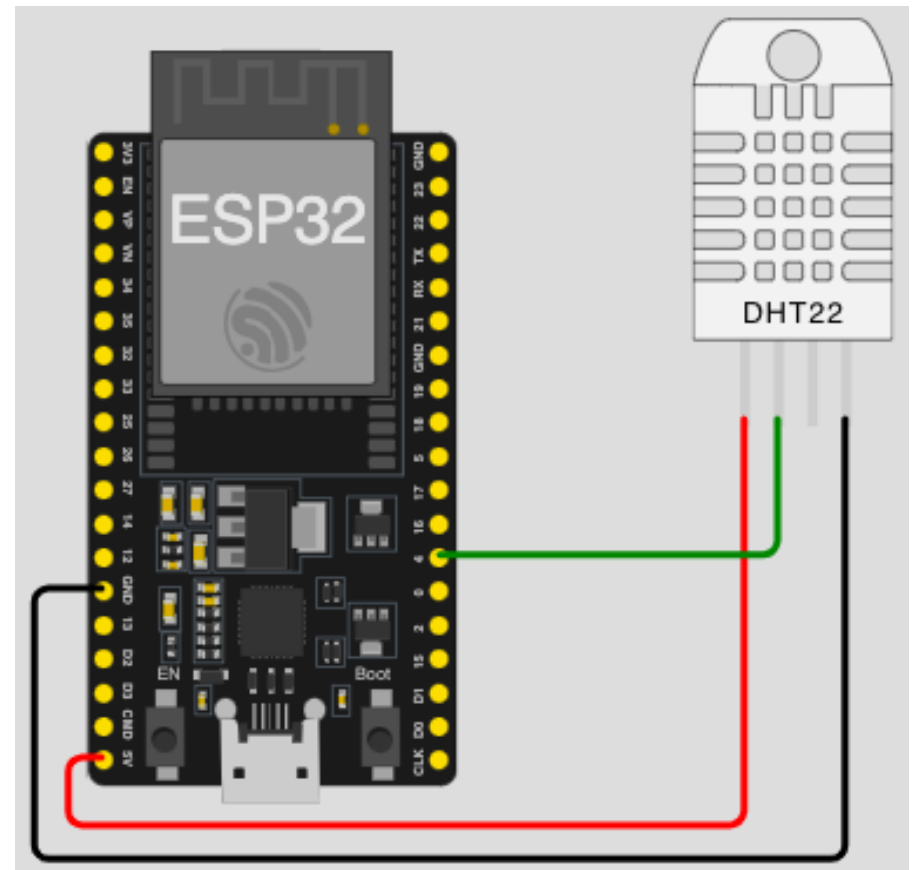
```
1  from machine import Pin, PWM, time_pulse_us
2  import time
3
4  trig = Pin(5, Pin.OUT)
5  echo = Pin(18, Pin.IN)
6  buzzer = PWM(Pin(15))
7
8  buzzer.freq(1000)
9  buzzer.duty_u16(0)
10
11 def get_distance():
12     trig.value(0)
13     time.sleep_us(2)
14     trig.value(1)
15     time.sleep_us(10)
16     trig.value(0)
17     try:
18         duration = time_pulse_us(echo, 1, 30000)
19         distance_cm = (duration / 2) / 29.1
20         return distance_cm
21     except OSError:
22         return None
23
24 while True:
25     distance = get_distance()
26     if distance is None:
27         print("No echo detected")
28         buzzer.duty_u16(0)
29     else:
30         print("Distance: {:.1f} cm".format(distance))
31         if distance < 100:
32             buzzer.duty_u16(32768)
33         else:
34             buzzer.duty_u16(0)
35     time.sleep(0.05)
```



<https://wokwi.com/projects/458308718129248257>

Example-6 : Cloud Data Logging

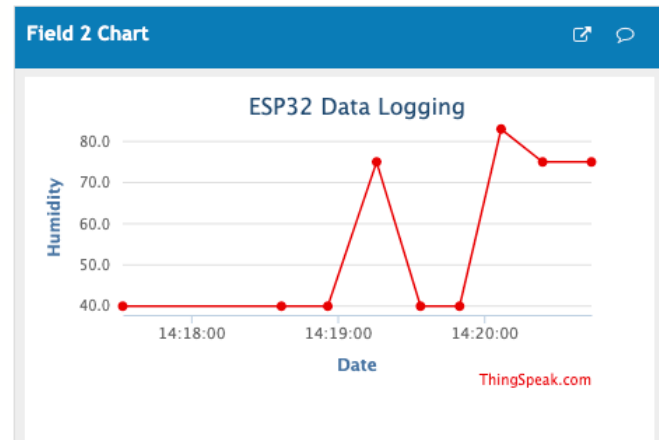
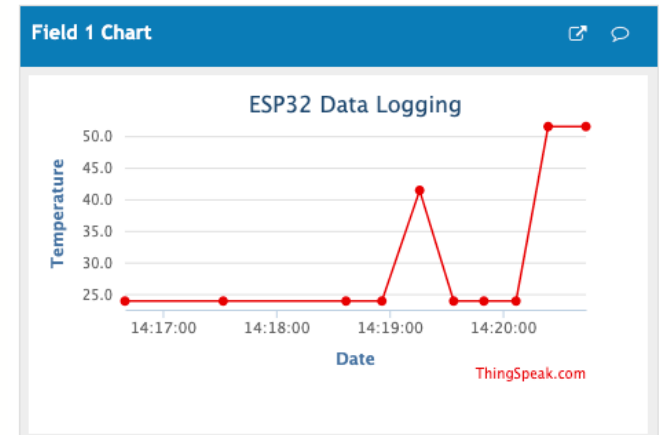
```
1  import network
2  import urequests
3  import time
4  import dht
5  from machine import Pin
6
7  SSID = "Wokwi-GUEST"
8  PASSWORD = ""
9
10 API_KEY = "I1T1KM3R12CRPJA0"
11
12 sensor = dht.DHT22(Pin(4))
13
14 def connect_wifi():
15     wlan = network.WLAN(network.STA_IF)
16     wlan.active(True)
17     wlan.connect(SSID, PASSWORD)
18
19     while not wlan.isconnected():
20         pass
21
22     print("WiFi Connected")
23
24 def read_dht():
25     try:
26         sensor.measure()
27         temp = sensor.temperature()
28         hum = sensor.humidity()
29         return temp, hum
30     except:
31         return None, None
```



<https://wokwi.com/projects/458819372056379393>

Example-6 : Cloud Data Logging (Cont...)

```
33 def send_to_thingspeak(temp, hum):
34     url = "http://api.thingspeak.com/update?api_key={}&field1={}&field2={}".format(API_KEY, temp, hum)
35
36     try:
37         response = urequests.get(url)
38         print("Sent -> Temp:", temp, "Hum:", hum, "| Response:", response.text)
39         response.close()
40     except:
41         print("Error sending data")
42
43 connect_wifi()
44
45 while True:
46     temperature, humidity = read_dht()
47
48     if temperature is not None and humidity is not None:
49         send_to_thingspeak(temperature, humidity)
50     else:
51         print("Sensor error")
52
53     time.sleep(15)
```



```
WiFi Connected
Sent -> Temp: 24.0 Hum: 40.0 | Response: 2
Sent -> Temp: 24.0 Hum: 40.0 | Response: 3
Sent -> Temp: 24.0 Hum: 40.0 | Response: 4
Sent -> Temp: 41.5 Hum: 75.0 | Response: 5
Sent -> Temp: 41.5 Hum: 75.0 | Response: 0
Sent -> Temp: -19.4 Hum: 75.0 | Response: 0
Sent -> Temp: 51.6 Hum: 75.0 | Response: 9
Sent -> Temp: 51.6 Hum: 75.0 | Response: 10
```

<https://thingspeak.mathworks.com/channels/3304783>